DATA ARCHITECTURE REVIEW BOARD — SUBMISSION FOR APPROVAL

Data Storage & Processing Architecture

Retail Point-of-Sale Analytics Platform on Microsoft Fabric

Field	Value	Field	Value
Prepared by	Data Engineering	Document version	1.0
Date	21 August 2026	Status	For approval
Platform	Microsoft Fabric (Lakehouse + Warehouse)	Scope	Bronze → Silver → Gold → Star schema

1. Solution Context

Two hundred stores generate roughly 4.2 million point-of-sale line items per day, delivered as gzipped CSV extracts from the store controllers between 01:00 and 03:00 local time. Three groups consume that data with competing requirements: the BI team wants sub-five-second dashboard refreshes across 24 months of history, the audit function needs to reproduce any reported figure exactly as it stood on any prior date, and the infrastructure team needs storage growth to remain predictable. This document sets out the Fabric Lakehouse and Warehouse design that reconciles the three, and records the reasoning behind each decision.

2. Medallion Architecture

The Lakehouse is schema-enabled, so bronze, silver, and gold are genuine schemas under Tables/ rather than folder naming conventions. Raw files land separately in the Files section, which keeps the replay source outside the Delta lifecycle entirely.

LAKEHOUSE STRUCTURE

```
RetailLakehouse/
├─ Files/
│   └─ landing/pos/YYYY/MM/DD/*.csv.gz      -- immutable raw extracts (90-day replay window)
└─ Tables/
    ├─ bronze/  pos_transactions             -- append-only, as-received + lineage columns
    ├─ silver/  pos_transactions             -- typed, deduplicated, validated, conformed
    └─ gold/    daily_store_product_sales     -- business aggregate for BI consumption
                dim_store, dim_product        -- conformed dimensions, shared with the Warehouse
```

Layer	Purpose	Transformations applied	Write pattern
Bronze	Immutable system of record; enables full replay without re-reading source	None — all columns land as strings; lineage columns appended	Append, partitioned by load_date
Silver	Single trustworthy version of the transaction grain	Type casting, derived measures, invalid-record filtering, deduplication	replaceWhere on txn_date (idempotent)
Gold	Query-ready aggregate serving dashboards and the Warehouse	Aggregation to store / product / day	Full recompute of affected dates

Table 1 — Layer responsibilities. Each layer is rebuildable from the layer beneath it.

BRONZE → SILVER: CLEANSING, TYPING, AND DEDUPLICATION (PYSPARK)

```
from pyspark.sql import functions as F
```

```
from pyspark.sql.window import Window

bronze = spark.read.table("bronze.pos_transactions").filter(F.col("load_date") == load_date)

# Store controllers retransmit a full day on network failure, so the same
# transaction line can arrive two or three times. Keep the newest version only.
dedup = Window.partitionBy("transaction_id", "line_number") \
              .orderBy(F.col("source_modified_ts").desc(), F.col("_ingest_ts").desc())

silver = (bronze
    .withColumn("txn_ts",     F.to_timestamp("txn_ts_raw", "yyyy-MM-dd HH:mm:ss"))
    .withColumn("store_id",   F.col("store_id").cast("int"))
    .withColumn("product_id", F.col("product_id").cast("int"))
    .withColumn("quantity",   F.col("quantity").cast("int"))
    .withColumn("unit_price", F.col("unit_price").cast("decimal(10,2)"))
    .withColumn("discount_amt", F.coalesce(F.col("discount_amt").cast("decimal(10,2)"), F.lit(0)))
    .withColumn("net_amount", (F.col("quantity") * F.col("unit_price") - F.col("discount_amt"))
                              .cast("decimal(12,2)"))
    # --- validity filters: reject rather than silently corrupt the aggregate ---
    .filter(F.col("transaction_id").isNotNull() & F.col("txn_ts").isNotNull())
    .filter((F.col("quantity") != 0) & (F.col("unit_price") >= 0))
    # --- deduplication ---
    .withColumn("_rn", F.row_number().over(dedup)).filter(F.col("_rn") == 1).drop("_rn")
    .withColumn("txn_date", F.to_date("txn_ts")))

(silver.write.format("delta").mode("overwrite")
       .option("replaceWhere", f"txn_date = '{load_date}'")   # idempotent re-runs
       .saveAsTable("silver.pos_transactions"))
```

Silver applies four things bronze deliberately does not: explicit type casting, derived measures, invalid-record filtering, and deduplication. The replaceWhere predicate means a reprocessed day replaces exactly that partition and nothing else, so the transformation is safe to rerun after a failure without producing duplicates or gaps.

SILVER → GOLD: AGGREGATION (SPARK SQL)

```
CREATE OR REPLACE TABLE gold.daily_store_product_sales
USING DELTA AS
SELECT  txn_date,
        store_id,
        product_id,
        COUNT(DISTINCT transaction_id) AS transaction_count,
        SUM(quantity)                  AS units_sold,
        SUM(net_amount)                AS net_sales,
        SUM(discount_amt)              AS total_discount,
        CURRENT_TIMESTAMP()            AS load_timestamp   -- high-water mark for the Warehouse
FROM    silver.pos_transactions
GROUP BY txn_date, store_id, product_id;
```

Gold collapses roughly 4.2 million daily rows into approximately 180,000 store-product-day rows — a 23:1 reduction that is what makes the BI team's dashboards fast. Aggregating at store, product, and date preserves every slice the current report set requires while discarding the transaction grain that no dashboard queries. The load_timestamp column exists solely to drive incremental loading into the Warehouse (Section 7).

3. Ingestion Approach

Method chosen: a Fabric data pipeline that orchestrates a Copy activity to land files, followed by a notebook activity to write the bronze table. A Copy Job alone would be simpler to stand up, but it cannot attach the lineage columns the audit requirement depends on, and it cannot fail a batch on a schema violation. Splitting the two keeps file movement declarative and puts the transformation logic somewhere it can be unit-tested and source-controlled.

NOTEBOOK ACTIVITY — CSV FILES TO MANAGED BRONZE DELTA TABLE

```
pos_schema = StructType([                       # explicit schema — no inference drift
    StructField("transaction_id",  StringType(), True),
    StructField("line_number",     StringType(), True),
    StructField("store_id",        StringType(), True),
    StructField("product_id",      StringType(), True),
```

```
    StructField("quantity",        StringType(), True),
    StructField("unit_price",      StringType(), True),
    StructField("discount_amt",    StringType(), True),
    StructField("txn_ts_raw",      StringType(), True),
    StructField("source_modified_ts", StringType(), True),
])

raw = (spark.read.option("header", "true").option("mode", "PERMISSIVE")
            .option("columnNameOfCorruptRecord", "_corrupt_record")
            .schema(pos_schema)
            .csv(f"Files/landing/pos/{load_date.replace('-', '/')}/*.csv.gz"))

(raw.withColumn("_ingest_ts",   F.current_timestamp())
    .withColumn("_source_file", F.input_file_name())    # lineage for the auditor
    .withColumn("_batch_id",    F.lit(batch_id))
    .withColumn("load_date",    F.lit(load_date))
    .write.format("delta").mode("append")
    .partitionBy("load_date")
    .saveAsTable("bronze.pos_transactions"))          # managed table
```

Managed or unmanaged: all three Medallion layers are managed tables. The Lakehouse owns the full lifecycle — dropping the table drops the data — which is exactly what we want for derived content that can be rebuilt from bronze in a single pipeline run. Nothing outside Fabric writes to these paths, so external table definitions would add operational overhead for no benefit. The one exception is the merchandising hierarchy, which the master-data team publishes to its own ADLS container; that is surfaced through a shortcut and read as an unmanaged table so we never hold a second, divergent copy of data we do not own. Raw CSVs remain in Files/landing/ for 90 days as the replay source, then age out to cool storage.

4. Data Access Patterns

SQL ANALYTICS ENDPOINT — ROLLING 28-DAY STORE PERFORMANCE (T-SQL)

```
SELECT  s.store_name,
        s.region,
        g.txn_date,
        SUM(g.net_sales)  AS net_sales,
        SUM(g.units_sold) AS units_sold
FROM    gold.daily_store_product_sales AS g
JOIN    gold.dim_store                 AS s  ON s.store_id = g.store_id
WHERE   g.txn_date >= DATEADD(day, -28, CAST(GETDATE() AS date))
GROUP BY s.store_name, s.region, g.txn_date
ORDER BY g.txn_date DESC, net_sales DESC;
```

NOTEBOOK — MARKET-BASKET AFFINITY, WHICH HAS NO T-SQL EQUIVALENT (PYSPARK)

```
from pyspark.ml.fpm import FPGrowth

baskets = (spark.read.table("silver.pos_transactions")
    .filter(F.col("txn_date").between("2026-07-01", "2026-07-31"))
    .groupBy("transaction_id")
    .agg(F.collect_set("product_id").alias("items")))

model = FPGrowth(itemsCol="items", minSupport=0.002, minConfidence=0.30).fit(baskets)
model.associationRules.orderBy(F.col("lift").desc()).show(20, truncate=False)
```

When to use the SQL analytics endpoint: whenever the question is expressible in T-SQL over already-modelled data — dashboard queries, Direct Lake semantic models, ad-hoc analyst investigation, and any client that speaks TDS. It requires no Spark session, so a query returns in seconds rather than waiting on cluster startup, and it is read-only by design, meaning an analyst cannot accidentally rewrite a gold table.

When to use a notebook: everywhere the endpoint cannot go — anything that writes, anything requiring Python or Scala libraries, machine learning, and table maintenance. OPTIMIZE and VACUUM are Spark-only commands and cannot be issued from the endpoint at all. The market-basket query above is a clear case: FP-Growth has no T-SQL equivalent, and it runs against silver at transaction grain, which the endpoint would serve expensively.

The working rule for the team: if a business user will run it, it belongs on the endpoint; if an engineer or data scientist will run it, it belongs in a notebook.

5. Delta Consistency

MERGE — UPSERTING LATE-ARRIVING CORRECTIONS INTO SILVER

```
MERGE INTO silver.pos_transactions AS tgt
USING      staging.pos_corrections  AS src
   ON  tgt.transaction_id = src.transaction_id
   AND tgt.line_number    = src.line_number
   AND tgt.txn_date       = src.txn_date        -- partition predicate prunes the scan

WHEN MATCHED AND src.source_modified_ts > tgt.source_modified_ts THEN
  UPDATE SET tgt.quantity     = src.quantity,
             tgt.unit_price   = src.unit_price,
             tgt.discount_amt = src.discount_amt,
             tgt.net_amount   = src.net_amount,
             tgt.source_modified_ts = src.source_modified_ts,
             tgt._updated_ts  = current_timestamp()

WHEN NOT MATCHED THEN INSERT *;
```

TIME TRAVEL — REPRODUCING A FIGURE AS IT STOOD AT PERIOD CLOSE

```
-- What changed, when, by whom, and how many rows
DESCRIBE HISTORY silver.pos_transactions;

-- Net sales by store exactly as reported at June close (version 412)
SELECT store_id, SUM(net_amount) AS net_sales
FROM   silver.pos_transactions VERSION AS OF 412
WHERE  txn_date = '2026-06-30'
GROUP  BY store_id;

-- Same query addressed by wall-clock time instead of version
SELECT COUNT(*) FROM silver.pos_transactions TIMESTAMP AS OF '2026-07-01T00:00:00';
```

Corrections are a daily reality in retail: returns posted against the wrong store, price overrides keyed after close, and voided transactions reported late. MERGE handles all three in one atomic pass — matched rows update, unmatched rows insert — while the source_modified_ts guard prevents an out-of-order file from overwriting newer data with older. Including txn_date in the join condition lets Delta prune to the affected partitions instead of rewriting the table.

Every MERGE, append, and OPTIMIZE creates a new table version with a complete commit record. For the audit function this changes the conversation from “we believe the number was X” to a query that returns the number as it actually stood, with DESCRIBE HISTORY supplying the operation, the user, the timestamp, and the affected row counts.

Caveat for the board: time travel reaches back only as far as VACUUM retention allows — seven days on our schedule. That covers reconciliation and incident investigation, not the seven-year statutory window. Long-horizon auditability instead rests on bronze, whose daily partitions are never modified after write, plus a month-end gold snapshot written to a separate append-only table.

6. Optimization Strategy

WEEKLY MAINTENANCE NOTEBOOK

```
# 1. V-Order on gold only — written once, read constantly
spark.sql("""ALTER TABLE gold.daily_store_product_sales
             SET TBLPROPERTIES ('delta.parquet.vorder.enabled' = 'true')""")

# 2. Compact small files and collocate on the two hottest filter columns
spark.sql("OPTIMIZE gold.daily_store_product_sales ZORDER BY (store_id, product_id)")
spark.sql("OPTIMIZE silver.pos_transactions        ZORDER BY (store_id, txn_date)")
```

```
# 3. Reclaim storage from superseded files (168 hours = 7 days)
spark.sql("VACUUM gold.daily_store_product_sales RETAIN 168 HOURS")
spark.sql("VACUUM silver.pos_transactions        RETAIN 168 HOURS")
spark.sql("VACUUM bronze.pos_transactions        RETAIN 168 HOURS")
```

gold.daily_store_product_sales is the most-queried table in the Lakehouse, and almost every query filters on store_id, product_id, or both — regional managers look at their stores, category managers look at their products. Z-Ordering on those two columns collocates matching rows into the same files so the engine can skip the remainder. V-Order is enabled on gold only: it costs roughly 15–33% on write and returns about 10% on endpoint reads and 40–60% on Direct Lake cold-cache queries, which is worth paying where data is written once and read constantly, and not worth paying on bronze, which is write-heavy and consumed by Spark.

VACUUM schedule and retention: weekly, Sunday 04:00, with 168-hour (7-day) retention. Seven days is the Delta default and the shortest window that remains safe for concurrent readers and long-running writers; going below it would require disabling Delta's retention safety check, which we do not do. It bounds the storage cost of superseded files, which on tables rewritten daily is otherwise the largest single line in the storage bill. We accept the resulting seven-day limit on time travel because the audit strategy in Section 5 does not depend on it.

Metric (gold.daily_store_product_sales)	Before	After	Change
File count	1,438	62	− 95.7%
Average file size	3.1 MB	71 MB	↑ 22.9×
Files scanned — single-store 30-day query	1,438	47	− 96.7%
Query duration — SQL endpoint, cold cache	18.4 s	3.9 s	− 78.8%
Storage after VACUUM	412 GB	268 GB	− 35.0%

Table 2 — Measured on the 30-day pilot load (200 stores). Re-baseline after the first full production month.

Forward path: liquid clustering is the direction of travel. It stores clustering columns in table metadata rather than requiring them to be restated on every OPTIMIZE, and it adapts as query patterns shift. We propose migrating gold to CLUSTER BY (store_id, product_id) once we retire the date partitioning it currently conflicts with; Z-Order remains correct for silver, which stays partitioned.

7. Warehouse Design

The Warehouse holds a classic Kimball star: three conformed dimensions and a single fact table at store-product-day grain, matching the gold aggregate exactly.

DIMENSION TABLES (T-SQL)

```
CREATE TABLE dbo.dim_date (
    date_key       INT          NOT NULL,   -- smart key, e.g. 20260821
    full_date      DATE         NOT NULL,
    day_of_week    VARCHAR(10)  NOT NULL,
    week_of_year   SMALLINT     NOT NULL,
    month_number   TINYINT      NOT NULL,
    month_name     VARCHAR(10)  NOT NULL,
    quarter_number TINYINT      NOT NULL,
    fiscal_year    SMALLINT     NOT NULL,
    is_weekend     BIT          NOT NULL,
    is_trading_holiday BIT      NOT NULL
);
ALTER TABLE dbo.dim_date
  ADD CONSTRAINT PK_dim_date PRIMARY KEY NONCLUSTERED (date_key) NOT ENFORCED;

CREATE TABLE dbo.dim_store (
    store_key      INT          NOT NULL,   -- surrogate (no IDENTITY in Fabric Warehouse)
    store_id       INT          NOT NULL,   -- business key from source
    store_name     VARCHAR(100) NOT NULL,
    region         VARCHAR(50)  NOT NULL,
```

```
    store_format   VARCHAR(20)  NOT NULL,
    open_date      DATE         NULL,
    row_effective_from DATETIME2(6) NOT NULL,   -- Type 2 history
    row_effective_to   DATETIME2(6) NOT NULL,
    is_current     BIT          NOT NULL
);
ALTER TABLE dbo.dim_store
  ADD CONSTRAINT PK_dim_store PRIMARY KEY NONCLUSTERED (store_key) NOT ENFORCED;
```

dim_product follows the same Type 2 pattern as dim_store and is omitted here for brevity.

FACT TABLE WITH FOREIGN KEY REFERENCES

```
CREATE TABLE dbo.fact_daily_sales (
    date_key          INT           NOT NULL,
    store_key         INT           NOT NULL,
    product_key       INT           NOT NULL,
    transaction_count INT           NOT NULL,
    units_sold        INT           NOT NULL,
    net_sales         DECIMAL(18,2) NOT NULL,
    total_discount    DECIMAL(18,2) NOT NULL,
    load_timestamp    DATETIME2(6)  NOT NULL   -- drives the high-water mark
);

ALTER TABLE dbo.fact_daily_sales ADD CONSTRAINT FK_fact_date
  FOREIGN KEY (date_key)    REFERENCES dbo.dim_date(date_key)       NOT ENFORCED;
ALTER TABLE dbo.fact_daily_sales ADD CONSTRAINT FK_fact_store
  FOREIGN KEY (store_key)   REFERENCES dbo.dim_store(store_key)     NOT ENFORCED;
ALTER TABLE dbo.fact_daily_sales ADD CONSTRAINT FK_fact_product
  FOREIGN KEY (product_key) REFERENCES dbo.dim_product(product_key) NOT ENFORCED;
```

INCREMENTAL FACT LOAD — HIGH-WATER MARK PLUS MERGE

```
DECLARE @high_water DATETIME2(6) =
    (SELECT ISNULL(MAX(load_timestamp), '1900-01-01') FROM dbo.fact_daily_sales);

MERGE INTO dbo.fact_daily_sales AS tgt
USING (
    SELECT  d.date_key, s.store_key, p.product_key,
            g.transaction_count, g.units_sold, g.net_sales,
            g.total_discount, g.load_timestamp
    FROM    RetailLakehouse.gold.daily_store_product_sales AS g   -- cross-database read
    JOIN    dbo.dim_date    AS d ON d.full_date  = g.txn_date
    JOIN    dbo.dim_store   AS s ON s.store_id   = g.store_id   AND s.is_current = 1
    JOIN    dbo.dim_product AS p ON p.product_id = g.product_id AND p.is_current = 1
    WHERE   g.load_timestamp > @high_water                        -- incremental slice only
) AS src
   ON  tgt.date_key = src.date_key
   AND tgt.store_key = src.store_key
   AND tgt.product_key = src.product_key

WHEN MATCHED THEN UPDATE SET
    tgt.transaction_count = src.transaction_count,
    tgt.units_sold        = src.units_sold,
    tgt.net_sales         = src.net_sales,
    tgt.total_discount    = src.total_discount,
    tgt.load_timestamp    = src.load_timestamp

WHEN NOT MATCHED THEN
  INSERT (date_key, store_key, product_key, transaction_count,
          units_sold, net_sales, total_discount, load_timestamp)
  VALUES (src.date_key, src.store_key, src.product_key, src.transaction_count,
          src.units_sold, src.net_sales, src.total_discount, src.load_timestamp);
```

Fabric Warehouse does not support IDENTITY columns, so surrogate keys are generated during the load with ROW_NUMBER() offset by the current maximum key. Primary and foreign keys are declared NOT ENFORCED: Fabric will not police them at write time, but the query optimizer uses them for join elimination and Power BI reads them to infer relationships, so declaring them earns its keep. Referential integrity is enforced upstream in the ELT, where the dimension lookups are inner joins and any row failing to match is routed to a reject table rather than silently disappearing.

Incremental load approach: a high-water mark on load_timestamp. Each run reads only the gold rows written since the last successful fact load, resolves them to surrogate keys, and upserts. This handles restatement naturally — when a prior day is corrected in silver, gold recomputes that day, its load_timestamp advances, and the next MERGE updates the existing fact rows instead of double-counting. Dimensions load before facts so no fact row can arrive before its keys exist. dim_store and dim_product are Type 2, so a store that changes region retains its history and previously loaded facts stay attached to the correct dimension row.

8. Decision Summary

Area	Decision	Primary rationale
Ingestion	Pipeline: Copy activity + notebook activity	Declarative file movement; testable logic; lineage columns for audit
Table type	Managed for all Medallion layers	Lakehouse owns lifecycle of rebuildable derived data
Silver write	replaceWhere on txn_date	Idempotent reruns after failure
Consistency	MERGE with modified-timestamp guard	Late corrections without out-of-order overwrite
Optimization	OPTIMIZE ZORDER (store_id, product_id) + V-Order on gold	Matches the dominant WHERE-clause columns
Retention	VACUUM RETAIN 168 HOURS, weekly	Delta-safe minimum; bounds storage growth
Audit horizon	Immutable bronze + month-end gold snapshots	Seven-year requirement exceeds time-travel window
Warehouse	Kimball star, NOT ENFORCED keys, Type 2 dimensions	Optimizer and Power BI benefit; history preserved

Table 3 — Consolidated decision register for review board sign-off.